Stage-4

计02 刘明道 2020011156

// Overview

```
git diff stage-3 stage-4
```

```
 .gitignore                      |  4 +++-
 .gitmodules                     |  2 +-
 arg_gen.py                      | 34 +++++++++++++++++++++++++++++++++
 backend/dataflow/basicblock.py  |  2 ++
 backend/dataflow/cfg.py         |  3 +++
 backend/dataflow/cfgbuilder.py  | 11 ++++++++++-
 backend/dataflow/loc.py         |  7 +++++++
 backend/reg/bruteregalloc.py    | 91 ++++++++++++++++++++++++++++++++
```

```
class Parameter(Declaration):
    def __init__(self, var_t: TypeLiteral, ident: Identifier):
    ...

class Call(Expression):
    def __init__(self, ident: Identifier, arg_list: List[Expression]):
    ...
```

也在 `frontend.ast.visitor` 中放置相应的方法。

给 `Program` 节点重载自增，用于添加函数

```
class Program(ListNode["Function"]):
    def __iadd__(self, other: List[Union[Function, Declaration]]):
        self.children += other
        return self
```

修改文法

修改 `frontend.ply_parser` 增加对函数声明的支持。

- `program` 的定义，可也推出空程序，以及函数和全局变量声明的序列。

  ```
  def p_program_empty(p):
      """
      program : empty
      """
      p[0] = Program()


  def p_program_component(p):
      """
      program : program function
      program : program declaration Semi  # 下一节实现全局变量用到，先写在这
      """
      p[1] += [p[2]]
      p[0] = p[1]
  ```

- 对于函数
  - 定义参数和参数列表

    ```
    def p_function_parameter_definition(p):
        """
        parameter : type Identifier
        """
        p[0] = [Parameter(*p[1:])]

    def p_function_parameter_list_empty(p):
        """
        parameter_list : empty
        """
        p[0] = []
    ```

```
def p_function_parameter_list_single(p):
    """
    parameter_list : parameter
    """
    p[0] = p[1]

def p_function_parameter_list_component(p):
    """
    parameter_list : parameter Comma parameter_list
    """
    p[0] = p[1] + p[3]
```

- 定义函数的声明以及定义

```
def p_function_delc_def(p):  # 定义函数
    """
    function : type Identifier LParen parameter_list RParen LBrace block RBrace
    """
    p[0] = Function(p[1], p[2], p[7], p[4])

def p_function_delc_only(p):  # 仅声明
    """
    function : type Identifier LParen parameter_list RParen Semi
    """
    p[0] = Function(p[1], p[2], NULL, p[4])
```

- 对于函数调用，参照规范定义 `expression_list` 和 `postfix`

```
def p_call_expression_list_empty(p):
    """
    expression_list : empty
    """
    p[0] = []

def p_call_expression_list_single(p):
    """
    expression_list : expression
    """
    p[0] = [p[1]]

def p_call_expression_list_component(p):
    """
    expression_list : expression_list Comma expression
    """
    p[0] = p[1] + [p[3]]

def p_postfix_call(p):
    """
    postfix : Identifier LParen expression_list RParen
    """
    p[0] = Call(p[1], p[3])
```

符号解析

首先是修改对 `Program` 的访问，现在需要遍历全部的函数

```
def visitProgram(self, program: Program, ctx: ScopeStack) -> None:
    ...
    for component in program:
        assert ctx.isGlobalScope()
        component.accept(self, ctx)
```

重点是访问 `Function` 节点。

- 首先构建该函数的符号
- 检查是否有符号冲突
  - 如果冲突的是函数，则暂时先不管，如果冲突的是（全局）变量，则报错
  - 没有冲突则声明这个符号
- 如果函数只有声明而没有定义，到这里就 `return`
- 如果函数有定义，那么调用 `FuncSymbol.define_function` 检查是否有重定义，具体实现为

  ```
  def define_function(self):
      if self.defined:
          raise DecafFuncMultipleDefinitionError(self.name)
      self.defined = True
  ```

- 建立局部作用域，访问所有参数（实现为直接调用 `visitDeclaration` ），然后逐一访问 `body` 中的语句。
  - 这里没有使用 `body.accept` 是因为 `visitBlock` 再次创建作用域，造成对与参数同名变量的声明无法检测

具体实现代码如下

```
    def visitFunction(self, func: Function, ctx: ScopeStack) -> None:
        # Check identifier conflict
        sym = FuncSymbol(func.ident.value, func.ret_t.type, ctx.currentScope())
        for param in func.param_list:
            sym.addParaType(param.var_t.type)

        potential_sym = ctx.lookup(func.ident.value)
        if ctx.findConflict(func.ident.value):
            if not (isinstance(potential_sym, FuncSymbol) and potential_sym == sym):
                raise DecafDeclConflictError(func.ident.value)
            sym = potential_sym
        else:
            ctx.declare(sym)

        func.setattr('symbol', sym)
        assert ctx.isGlobalScope()
        if func.body is NULL:  # function decl only
            return
        sym.define_function()
        with ctx.local():
            for param in func.param_list:
                param.accept(self, ctx)
            # Visit body statements.
```

```
        # Note that visit the block directly will generate a new scope
        for stmt in func.body.children:
            stmt.accept(self, ctx)
```

同时，实现了访问 `Call` 节点。

- 检查函数符号是否已经定义
- 检查参数个数是否相同（因为 miniDecaf 只有一种类型）
- 设置对应的函数符号
- 访问所有参数

```
    def visitCall(self, call: Call, ctx: ScopeStack) -> None:
        func: FuncSymbol = ctx.lookup(call.ident.value)
        # Check if function is defined
        if func is None or func.isFunc is False:
            raise DecafUndefinedFuncError(call.ident.value)

        # Check if param_list match
        if len(call.arg_list) != func.parameterNum:
            raise DecafBadFuncCallError(
                f"Expecting {func.parameterNum} parameters, got {len(call.arg_list)}"
            )

        call.ident.setattr('symbol', func)
        for arg in call.arg_list:
            arg.accept(self, ctx)
```

生成 TAC

TAC 指令定义

与文档不同，这里仅定义函数调用的 `Call` 指令，参数的传递全部由后端实现

```
class Call(TACInstr):
    def __init__(self, func_label: Label, dst: Temp, param_temps: List[Temp]):
        super(Call, self).__init__(InstrKind.SEQ, [dst], param_temps, func_label)

    def __str__(self) -> str:
        return f"{self.dsts[0]} = CALL {self.label.name} ({self.srcs})"
    ...
```

而对 `Call` 的访问定义为

```
def visitCall(self, func_label: Label, dst: Temp, param_temp: List[Temp]):
    self.func.add(Call(func_label, dst, param_temp))
```

TAC 指令生成

在 `transform` 时，要访问程序的所有函数，而不只是 `main` 函数

```
pw = ProgramWriter(
    program.functions().values(),
)

for name, func in program.functions().items():
    if func.body is NULL:
        continue
    mv = pw.visitFunc(name, len(func.param_list))
    func.accept(self, mv)
    mv.visitEnd()
```

重点是为新的 `AST` 节点生成 TAC

- 函数
  - 访问其所有参数，然后访问 `body`
- 参数
  - 首先使用 `visitDeclaration` 分配相应的临时变量
  - 然后将该临时变量绑定到函数上，便于后续传参时使用
- 函数调用
  - 遍历所有参数，为参数求值
  - 分配一个储存返回值的临时变量

具体实现如下

```
def visitFunction(self, func: Function, mv: FuncVisitor) -> None:
    for p in func.param_list:
        p.accept(self, mv)
    func.body.accept(self, mv)

def visitParameter(self, that: Parameter, mv: FuncVisitor) -> None:
    temp = self.visitDeclaration(that, mv)
    mv.func.addArgTemp(temp)

def visitCall(self, call: Call, mv: FuncVisitor) -> None:
    # Evaluate Argument Expressions
    param_temp = []
    for arg_expr in call.arg_list:
        arg_expr.accept(self, mv)
        assert arg_expr.getattr('val')
        param_temp += [arg_expr.getattr('val')]

    # Assign Return Value
    ret = mv.freshTemp()
    call.setattr('val', ret)
    func_label = mv.ctx.getFuncLabel(call.ident.value)
    mv.visitCall(func_label, ret, param_temp)
```

生成 RISC-V 汇编

修改 CFG 的生成

阅读代码后发现，原先的寄存器分配策略中，会在每个基本块中重新进行寄存器分配。

将 `Call` 这一 TAC 指令单独作为一个基本块。这样就无需额外考虑调用者保存寄存器的存储，同时也不会出现传参时的寄存器冲突。

```
def buildFrom(self, seq: list[TACInstr]):
    for item in seq:
        if item.isLabel():
            ...
        else:
            if isinstance(item, Riscv.RCall):
                bb = BasicBlock(BlockKind.CONTINUOUS, len(self.bbs), self.currentBBLabel, self.buf)
                self.save(bb)
                self.buf.append(Loc(item))
                bb = BasicBlock(BlockKind.CALL, len(self.bbs), self.currentBBLabel, self.buf)
                self.save(bb)
                continue
    ...
```

修改寄存器的分配

这一部分描述在 `backend.reg.bruteregalloc` 中进行的修改。

函数参数的加载

刚进入函数调用时，前 8 个参数储存在寄存器中，而进入每个基本块时，我们假设临时变量都储存在栈上，因此这里将前 8 个参数存到栈上。具体实现如下

```
def accept(self, graph: CFG, info: SubroutineInfo) -> None:
    ...
    # bind (actually stash) A0 ~ A7
    # other args are marked in `RiscvSubroutineEmitter.offsets`
    if not self.paramsBound:
        self.paramsBound = True
        for temp, reg in zip(subEmitter.info.argTemps, Riscv.ArgRegs):
            self.bind(temp, reg)
    if len(graph) > 0:
        for tempindex in graph.nodes[0].liveIn:
            if tempindex in self.bindings:
                subEmitter.emitStoreToStack(self.bindings.get(tempindex))
    self.restoreBindings()
    ...
```

函数参数的传递

为基本块分配寄存器时，到了 `Call` 所在基本块，需要准备传参。

- 首先对第 9 个及后续的参数压栈传参。下拉栈指针，将变量从栈上取出，存放到 `a0` 中，然后再 `a0` 中放到栈上的相应位置。
  - 这里调用了 `subEmitter.adjustSP(-4 * remain_args_count)`

    这是为了在栈指针下拉后，对原先保存在栈上的变量，相对栈指针的偏移量也发生了变化。
- 接下来存将前 8 个参数从栈上取出，加载到对应的 `a0 ~ a7` 中

- 发出 RISC-V 的 `call` 伪指令
- 收起用于传参的栈
- 将返回值对应的临时变量与 `a0` 绑定

具体实现如下

```
if bb.kind == BlockKind.CALL:
    assert len(bb.locs) == 1
    loc = bb.locs[0]
    call = loc.instr

    # Prepare other arguments
    reg = Riscv.T0
    remain_args_count = len(call.srcs) - 8
    if remain_args_count > 0:
        subEmitter.emitNative(Riscv.SPAdd(-4 * remain_args_count))
        subEmitter.adjustSP(-4 * remain_args_count)
        for idx, temp in enumerate(call.srcs[8:]):  # to regs
            subEmitter.emitComment(
                "  PARAM BORROW: allocate {} to {}  (read: {}):".format(
                    str(temp), str(reg), str(True)
                )
            )
            subEmitter.emitLoadFromStack(reg, temp)
            assert not reg.occupied, f"{reg}, {reg.temp}"
            self.bind(temp, reg)
            subEmitter.emitNative(Riscv.NativeStoreWord(reg, Riscv.SP, 4 * idx))
            self.unbind(temp)

    # Prepare arguments [0, 7]
    for temp, reg in zip(call.srcs, Riscv.ArgRegs):
        subEmitter.emitComment(
            "  PARAM DIRECT: allocate {} to {}  (read: {}):".format(
                str(temp), str(reg), str(True)
            )
        )
        subEmitter.emitLoadFromStack(reg, temp)
        assert not reg.occupied
        self.bind(temp, reg)

    # Function call
    subEmitter.emitComment(str(call))
    subEmitter.emitNative(call.toNative(call.dsts, call.dsts))

    # Clean up stack
    if remain_args_count > 0:
        subEmitter.emitNative(Riscv.SPAdd(4 * remain_args_count))
        subEmitter.adjustSP(4 * remain_args_count)

    # Store return value
    if len(call.srcs) > 0:
        self.unbind(call.srcs[0])
    self.bind(call.dsts[0], Riscv.A0)
    subEmitter.emitStoreToStack(Riscv.A0)
```

其余对基本块的访问基本保持原先的设计。

函数开头和结尾的生成

这部分主要叙述对 `backend.riscv.riscvasmemitter` 的修改。这里先列出我们设计的函数栈的模式

```
# Stack Structure
Arg (len - 1)
...
Arg 8
------------- `RiscvSubroutineEmitter.nextLocalOffset`
SPILLED REGS END
...
SPILLED REGS BEGIN
RETURN ADDRESS
CALLEE SAVE END
...
CALLEE SAVE BEGIN
------------- SP
```

在 `emitEnd` 中，函数的全部汇编代码被生成。在 `prologue` 中

- 储存 `ra` 和 在函数执行过程中被使用过的调用者保持寄存器
- 确定传入的第 9 个及之后的参数的位置，将它们在栈上的位置与临时变量的索引进行绑定
  - 这里有个问题
    - 压栈形式传入参数相对下拉后的栈指针的位置只有在翻译完 TAC 指令后才能确定（因为翻译过程中可能因为寄存器不够用而将寄存器 spill 到栈上）
    - 但翻译的过程中，首次使用这些寄存器时又需要确定它们的地址
  - 因此，我们另开字典保存这些变量的临时位置，也就是假设 `self.nextLocalOffset == 0` 时相对栈指针的位置

    ```
    self.argOffset: Dict[int, int] = {}
    for idx, temp in enumerate(self.info.argTemps[8:]):
        self.argOffset[temp.index] = idx * 4
    ```

    在 `emitLoadFromStack` 中，如果在 `self.offset` 中查不到寄存器的位置，那么再来查看 `self.argOffset` ，若能查到，则在 `self.buf` 中增加一条有标记的 `NativeLoadWord`

    ```
    def emitLoadFromStack(self, dst: Reg, src: Temp):
        if src.index not in self.offsets:
            if src in self.info.argTemps:
                offset = self.argOffset[src.index] - self.sp_offset
                self.buf.append(
                    Riscv.NativeLoadWord(dst, Riscv.SP, offset , src)
                )
                return
        ...
    ```

    这里 `offset` 减去 `self.sp_offset` ，就是减去上文提到的 `subEmitter.adjustSP` 设置的，用于在函数调用内部移动栈指针时，维护偏移量正确。

    在打印指令前，对首次访问这些临时变量的指令，调整 `offset`

```
for instr in self.buf:
    ...
    # Now the nextLocalOffset have been determined (considering possible spills)
    # update the positions of potential arguments
    if isinstance(instr, Riscv.NativeLoadWord) and instr.temp is not None:
        assert instr.temp.index in self.argOffset
        instr.offset += self.nextLocalOffset

    ...
```

- 函数调用结束后，恢复被调用者保存寄存器和 `ra`

这一节还对许多其他一些代码进行了修改。这些修改大多是一些工具方法或工具类，不是实现思路的重点。这里就不逐一指出了。

Step-10 全局变量

这一节要支持全局变量，主要工作是在变量读取和赋值时进行判断，如果是全局变量则需要从内存中加载值 / 将值写回内存；此外还需要进行文法的支持和相关汇编代码的生成。

修改文法

全局变量的解析文法在上一节中已经实现。

符号解析

在 `frontend/typecheck/namer.py` 中

- `visitDeclaration` 时，增加一条新规则，如果此时是 `GlobalScope`，则将 `VarSymbol` 的 `isGlobal` 置为 `True`；如果有返回值，一并放入 `VarSymbol`

  ```
  def visitDeclaration(self, decl: Declaration, ctx: ScopeStack) -> None:
      ...
      if ctx.isGlobalScope():
          var.isGlobal = True
          if decl.init_expr is not NULL:
              assert isinstance(decl.init_expr, IntLiteral)
              var.initValue = decl.init_expr.value
      ...
  ```

生成 TAC

保存全局变量信息

这时 ProgramWriter 不仅需要存储函数信息，还需要存储全局变量的名称和初值（如果有）

```python
class ProgramWriter:
    def __init__(
        self, funcs: List[Function],
        globalDecls: List[Tuple[str, Optional[int]]]
    ) -> None:
        ...
        self.globalDecls = globalDecls
```

`TACProg` 也进行类似的修改。

而这些信息则在 `TACGen.transform` 中给出

```python
class TACGen(Visitor[FuncVisitor, None]):
    ...
    def transform(self, program: Program) -> TACProg:
        ...
        pw = ProgramWriter(
            program.functions().values(),
            [
                (name, decl.getattr('symbol').initValue)
                for name, decl in program.globalDecls().items()
            ]
        )
```

全局变量地址的加载

对于全局变量来说，每次使用时不能直接从寄存器中取用，而是需要从全局变量的地址进行加载。因此新增以下三条 TAC 指令

```python
class LoadSymbolAddress(TACInstr):
    '''
    Load the address of `global_sym` to variable `dst`, similar to `la` in RISC-V
    '''
    def __init__(self, global_sym, dst: Temp):
        super(LoadSymbolAddress, self).__init__(InstrKind.SEQ, [dst], [])
        self.symbol = global_sym

class LoadWord(TACInstr):
    '''
    Load the word in `base` + `offset` to `dst` , similar to `lw` in RISC-V
    '''
    def __init__(self, dst: Temp, base: Temp, offset: int):
        super(LoadWord, self).__init__(InstrKind.SEQ, [dst], [base])
        self.offset = offset

class StoreWord(TACInstr):
    '''
    Store variable `src` to `base` + `offset`, similar to `sw` in RISC-V
    '''
    def __init__(self, src: Temp, base: Temp, offset: int):
        super(StoreWord, self).__init__(InstrKind.SEQ, [], [src, base])
        self.offset = offset
```

而在 FunctionVisitor 中也新增类似的访问（生成）方法

```python
    def visitLoadGlobal(self, global_sym: VarSymbol) -> Temp:
        '''
        Get a variable that stores the value of `global_sym`
        '''
        dst = self.freshTemp()
        self.func.add(LoadSymbolAddress(global_sym, dst))
        self.func.add(LoadWord(dst, dst, 0))  # reuse `dst`
        return dst

    def visitStoreGlobal(self, global_sym: VarSymbol, src: Temp) -> None:
        '''
        Store the value in `src` to `global_sym`
        '''
        dst = self.freshTemp()
        self.func.add(LoadSymbolAddress(global_sym, dst))
        self.func.add(StoreWord(src, dst, 0))
```

生成 TAC 序列

在有了 `FuncVisitor` 中的工具函数后，生成 TAC 序列的任务就很简单了。

- `visitIdentifier` 如果遇到全局变量，则调用前述的 `FuncVisitor.visitLoadGlobal` 获得该全局变量的值。
- `visitAssignment` 如果遇到全局变量，则调用前述的 `FuncVisitor.visitStoreGlobal` 将左操作数中的值保存到全局变量对应的地址中。

生成 RISC-V 汇编

- 首先在 `.data` 段和 `.bss` 段分别声明初始化的和未初始化的全局变量

```python
class RiscvAsmEmitter(AsmEmitter):
    def __init__(
        self,
        allocatableRegs: list[Reg],
        callerSaveRegs: list[Reg],
        globalDecls: List[Tuple[str, Optional[int]]]
    ) -> None:
        ...
        self.printer.println(".data")  # place initalized global var in .data
        for name, initial_val in filter(lambda x: x[1], globalDecls):
            self.printer.printDATAWord(name, initial_val)

        self.printer.println("")
        self.printer.println(".bss")  # place uninitalized global var in .bss
        for name, _ in filter(lambda x: not x[1], globalDecls):
            self.printer.printBSS(name, 4)

        self.printer.println("")
```

- 同时也在 `utils/riscv.py` 实现之前添加的三条 TAC 指令，并在 `RiscvAsmEmitter.RiscvInstrSelector` 中添加相应的访问函数。具体来说，`LoadSymbolAddress` ，`LoadWord`，`StoreWord` 分别翻译为 `la` ，`lw` ，`sw` 。

这就实现了全局变量支持。此外还创建了一些工具函数，这里就不逐一列举了。

思考题

Step-9 函数

1. MiniDecaf 的函数调用时参数求值的顺序是未定义行为。试写出一段 MiniDecaf 代码，使得不同的参数求值顺序会导致不同的返回结果。

```
// function definition
int foo(int a, int b) {
    return a - b;
}

int main() {
    int bar = 0;
    return foo(bar, bar = bar + 1);  // function call
}
```

对于 `foo(bar, bar = bar + 1)` 这一函数调用，若从右向左求值，则等价于

```
foo(1, 1);
```

若从左向右求值，则等价于

```
foo(0, 1);
```

此时，不同的求值顺序有不同的返回结果。

2. 为何 RISC-V 标准调用约定中要引入 callee-saved 和 caller-saved 两类寄存器，而不是要求所有寄存器完全由 caller/callee 中的一方保存？为何保存返回地址的 ra 寄存器是 caller-saved 寄存器？
   - 为何不是所有寄存器完全由 caller/callee 中的一方保存？
     - 如果所有寄存器都为 caller-saved，那么 caller 就必须保存所有使用到的通用寄存器，然而被 callee 使用的可能只是其中的一小部分，全部保存则每次函数调用的开销过大。而编译器可以选择将一些在函数调用后还需使用的值存入 callee-saved ，这样只有 callee 用到这些寄存器才需要在 callee 中保存和恢复这些值。
     - 为何不全部采用 callee-saved?
       - 这样 callee 会保并恢复全部用到的寄存器，开销过大。
       - 一些寄存器的值本身就是为函数调用准备的，在函数调用后可以直接丢弃，而并不需要在调用后恢复。如果将这些不需要恢复的值存入 caller-saved 寄存器再进行函数调用，就可以节约一些不必要的保存与恢复。例如传参时使用的 `a0~a7` 就是 caller-saved 寄存器。
       - 如果全部采用 callee-saved，返回值将无法通过通用寄存器传递（因为这会在函数调用前后修改返回值寄存器的值）；另外，某些寄存器在函数调用过程中会被修改，因此无法由 callee save ，如 `ra`
     - 由此可见，引入 callee-saved 和 caller-saved 两类寄存器是一种较为高效和方便的选择。
   - 为何保存返回地址的 ra 寄存器是 caller-saved 寄存器？
     - 在调用函数时，新的返回地址被 `jalr` 等指令保存到 `ra`， 因此在进入被调用函数后， `ra` 的值已经被修改，被调用者无法保存一个已经在调用过程中被修改的值。

Step-10 全局变量

1. 写出 `la v0, a` 这一 RiscV 伪指令可能会被转换成哪些 RiscV 指令的组合（说出两种可能即可）。
    1. 可能转换为

    ```
    auipc v0, delta[31:12] + delta[11]
    addi v0, v0, delta[11:0]
    ```

    其中 `delta` 表示符号 `a` 的地址相对当前指令地址的偏移量。

    2. 可能转换为

    ```
    auipc v0, delta[31:12] + delta[11]
    lw v0, delta[11:0](v0)
    ```

    其中 `delta` 为变量 `a` 的地址在 Global Offset Table 中的位置与当前指令地址的差。

    3. 观察 gcc 的一些行为后发现，假设 `a` 的地址与 `gp` 值的差可以被立即数偏移表达，那么可转换为

    ```
    addi v0, gp, delta_gp
    ```

    其中 `delta_gp` 表示符号 `a` 的地址相对 `gp` 值的偏移量。

    而假如偏移量过大，则会直接采用 1 所示的方式。